TP4 – OpenMP

Parallel Sections, Single, Master, Synchronization

Lab Report by Mounia BADDOU

---

1. Exercise 1: Work Distribution with Parallel Sections

1.1 Objective

The goal here is to initialize a large array of $N = 10^6$ random values, then use `#pragma omp sections` to compute three statistics in parallel: the sum, the maximum value, and the standard deviation. The constraint is that Section 3 must reuse the sum computed by Section 1 rather than recomputing it.

1.2 Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define N 1000000

int main() {
    double *A = malloc(N * sizeof(double));
    double sum = 0.0, mean = 0.0, stddev = 0.0, max;

    srand(0);
    for (int i = 0; i < N; i++)
        A[i] = (double)rand() / RAND_MAX;

    max = A[0];

    /* First parallel region: sum and max run in separate sections */
    #pragma omp parallel sections shared(sum, max)
    {
        #pragma omp section
        {
            double local_sum = 0.0;
            for (int i = 0; i < N; i++)
                local_sum += A[i];
            #pragma omp critical
            sum += local_sum;
        }

        #pragma omp section
        {
            double local_max = A[0];
            for (int i = 1; i < N; i++)
                if (A[i] > local_max) local_max = A[i];
            #pragma omp critical
            { if (local_max > max) max = local_max; }
        }
    }
    /* Implicit barrier here: sum is ready */
```

```c
    mean = sum / N;

    /* Second parallel region: stddev uses the already computed mean */
    #pragma omp parallel sections shared(stddev)
    {
        #pragma omp section
        {
            double local_sd = 0.0;
            for (int i = 0; i < N; i++)
                local_sd += (A[i] - mean) * (A[i] - mean);
            #pragma omp critical
            stddev += local_sd;
        }
    }
    stddev = sqrt(stddev / N);

    printf("Sum    = %f\n", sum);
    printf("Max    = %f\n", max);
    printf("StdDev = %f\n", stddev);

    free(A);
    return 0;
}
```

Listing 1: Exercise 1 – parallel sections for sum, max, and std dev

1.3 Output

```
Sum    = 500255.730712
Max    = 0.999999
StdDev = 0.288723
```

1.4 Discussion

The sum (≈ 500255) makes sense: with $N = 10^6$ uniform random values in $[0, 1]$, the expected sum is $N/2 = 500000$. The maximum is very close to 1 which is also expected for a million draws from a uniform distribution. The standard deviation of ≈ 0.2887 matches the theoretical value $1/\sqrt{12} \approx 0.2887$ for a uniform distribution on $[0, 1]$.

The key design point is the barrier between the two parallel regions. Without it, Section 3 would start computing the standard deviation before `sum` is finalized, which would give a wrong mean and therefore a wrong standard deviation. The implicit barrier at the end of the first `#pragma omp parallel sections` block guarantees the sum is ready before we compute the mean.

2. Exercise 2: Exclusive Execution: Master vs Single

2.1 Objective

Use `#pragma omp master` to have only the master thread initialize a matrix, `#pragma omp single` to print it, and parallelize the sum computation across all threads. The exercise also asks us to compare execution time with and without OpenMP.

2.2 Implementation

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000

void init_matrix(int n, double *A) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            A[i*n + j] = (double)(i + j);
}

double sum_matrix_parallel(int n, double *A) {
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < n*n; i++)
        sum += A[i];
    return sum;
}

int main() {
    double *A = (double*) malloc(N * N * sizeof(double));
    double sum;
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel
    {
        #pragma omp master
        {
            init_matrix(N, A);
        }
        #pragma omp barrier

        #pragma omp single
        {
            /* print_matrix would go here -- skipped for N=1000 */
        }
    }

    sum = sum_matrix_parallel(N, A);
    end = omp_get_wtime();
```

```c

    printf("Sum = %lf\n", sum);
    printf("Execution time = %lf seconds\n", end - start);

    free(A);
    return 0;
}
```

Listing 2: Exercise 2 – master, single, and parallel sum

2.3 Output

```
Running Exercise 2 (1 thread)
Sum = 999000000.000000
Execution time = 0.006055 seconds

Running Exercise 2 (4 threads)
Sum = 999000000.000000
Execution time = 0.003541 seconds
```

2.4 Discussion

The sum can be verified analytically. For a matrix where $A[i][j] = i+j$ with $0 \leq i, j < N$:

$$\sum_{i=0}^{N-1}\sum_{j=0}^{N-1}(i+j) = 2N \cdot \frac{N(N-1)}{2} = N^2(N-1) = 1000^2 \times 999 = 999\,000\,000$$

which matches exactly.

Going from 1 thread (6.055 ms) to 4 threads (3.541 ms) gives a speedup of about 1.71×. This is lower than the ideal 4× mainly because the matrix initialization is done serially by the master thread, and the `#pragma omp barrier` that follows ensures all threads wait before the sum begins. The serial initialization takes a significant fraction of the total time, which directly limits the overall speedup according to Amdahl's Law.

The difference between `master` and `single` is worth noting: `master` executes only on thread 0 with no implicit barrier afterward, while `single` can be executed by whichever thread arrives first and does include an implicit barrier at the end of the block.

3. Exercise 3: Load Balancing with Parallel Sections

3.1 Objective

Simulate three tasks of very different computational weights (light, moderate, heavy) using `#pragma omp sections`, measure total execution time, and think about how to optimize the workload distribution.

3.2 Implementation

```c
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define N 1000000

void task_light(int N) {
    double x = 0.0;
    for (int i = 0; i < N; i++)
        x += sin(i * 0.001);
}

void task_moderate(int N) {
    double x = 0.0;
    for (int i = 0; i < 5*N; i++)
        x += sqrt(i * 0.5) * cos(i * 0.001);
}

void task_heavy(int N) {
    double x = 0.0;
    for (int i = 0; i < 20*N; i++)
        x += sqrt(i * 0.5) * cos(i * 0.001) * sin(i * 0.0001);
}

int main() {
    double start = omp_get_wtime();

    #pragma omp parallel sections
    {
        #pragma omp section
        { task_light(N); }

        #pragma omp section
        { task_moderate(N); }

        #pragma omp section
        { task_heavy(N); }
    }

    double end = omp_get_wtime();
    printf("Execution time = %f seconds\n", end - start);
    return 0;
}
```

Listing 3: Exercise 3 – three tasks of different weights in parallel sections

3.3 Output

```
Execution time = 1.100860 seconds
```

3.4 Discussion

With `#pragma omp sections`, each section is assigned to one thread. The three tasks have very unequal workloads.

The problem with sections for this kind of workload is that the overall parallel region finishes when the *slowest* section finishes. Threads that complete their section early just sit idle waiting at the barrier. In this case, the total runtime is essentially determined by `task_heavy` alone, meaning the other two threads are wasted for most of the execution.

A better approach would be to break `task_heavy` into smaller pieces and distribute them with `schedule(dynamic)`, or to pipeline the tasks differently so that lighter tasks are grouped together while the heavy one runs alone. Another option is to not use sections at all and instead use a `parallel for` on the inner loops of each task with a dynamic schedule.

4. Exercise 4: Synchronization and Barrier Cost

4.1 Objective

Parallelize a dense matrix-vector multiplication in three versions:

- **Version 1:** implicit barrier (default `parallel for`)
- **Version 2:** `schedule(dynamic)` with `nowait`
- **Version 3:** `schedule(static)` with `nowait`

and measure CPU time, speedup, efficiency, and MFLOP/s for 1, 2, 4, 8, 16 threads.

4.2 Implementation

The matrix has dimensions $m \times n$ with $n = 40000$ columns and $m = 600$ rows. The total number of floating-point operations is $2 \times n \times m = 2 \times 40000 \times 600 = 48\,000\,000$ FLOPs.

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

/* Version 1: default parallel for with implicit barrier */
void dmvm_v1(int n, int m, double *lhs, double *rhs, double *mat) {
    #pragma omp parallel for schedule(static)
    for (int c = 0; c < n; ++c) {
        int offset = m * c;
        for (int r = 0; r < m; ++r)
            lhs[r] += mat[r + offset] * rhs[c];
    }
    /* implicit barrier here */
}

/* Version 3: static schedule + nowait, no barrier at end of loop
   */
void dmvm_v3(int n, int m, double *lhs, double *rhs, double *mat) {
    #pragma omp parallel
    {
        #pragma omp for schedule(static) nowait
```

```c
        for (int c = 0; c < n; ++c) {
            int offset = m * c;
            for (int r = 0; r < m; ++r)
                lhs[r] += mat[r + offset] * rhs[c];
        }
        /* no barrier: threads exit as soon as their chunk is done */
    }
}

int main(void) {
    const int n = 40000, m = 600;
    double *mat = malloc(n * m * sizeof(double));
    double *rhs = malloc(n * sizeof(double));
    double *lhs = malloc(m * sizeof(double));
    const long flops = 2L * n * m;

    for (int c = 0; c < n; ++c) {
        rhs[c] = 1.0;
        for (int r = 0; r < m; ++r)
            mat[r + c*m] = 1.0;
    }

    /* Version 1 timing */
    for (int r = 0; r < m; ++r) lhs[r] = 0.0;
    double t0 = omp_get_wtime();
    dmvm_v1(n, m, lhs, rhs, mat);
    double t1 = omp_get_wtime();
    printf("V1 time = %f s, MFLOPS = %f\n",
           t1-t0, flops/((t1-t0)*1e6));

    /* Version 3 timing */
    for (int r = 0; r < m; ++r) lhs[r] = 0.0;
    t0 = omp_get_wtime();
    dmvm_v3(n, m, lhs, rhs, mat);
    t1 = omp_get_wtime();
    printf("V3 time = %f s, MFLOPS = %f\n",
           t1-t0, flops/((t1-t0)*1e6));

    free(mat); free(rhs); free(lhs);
    return 0;
}
```

Listing 4: Exercise 4 – dmvm, Version 1 (implicit barrier) and Version 3 (static + nowait)

4.3 Raw Performance Data

4.3.1 Version 1 – Implicit Barrier

Table 1: Version 1 (implicit barrier) – performance vs. thread count

Threads	Time (s)	Speedup	Efficiency (%)	MFLOP/s
1	0.045242	1.00	100.0	1060.96
2	0.019500	2.32	116.0	2461.54
4	0.011612	3.90	97.4	4133.68
8	0.013182	3.43	42.9	3641.35
16	0.012465	3.63	22.7	3850.78

4.3.2 Version 3 – Static Schedule + nowait

Table 2: Version 3 (static + nowait) – performance vs. thread count

Threads	Time (s)	Speedup	Efficiency (%)	MFLOP/s
1	0.036894	1.00	100.0	1301.03
2	0.020663	1.79	89.3	2322.99
4	0.013413	2.75	68.8	3578.63
8	0.016111	2.29	28.6	2979.31
16	0.018777	1.97	12.3	2556.33

4.4 Charts

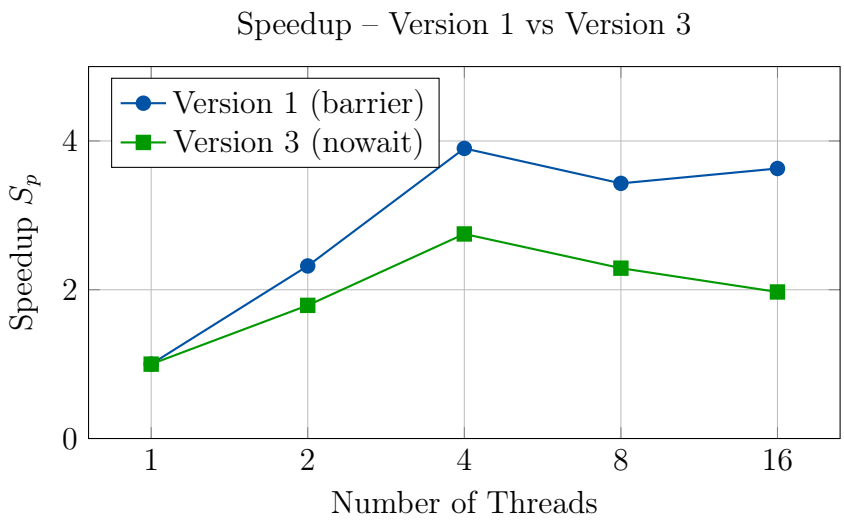

Figure 1: Speedup comparison between Version 1 and Version 3

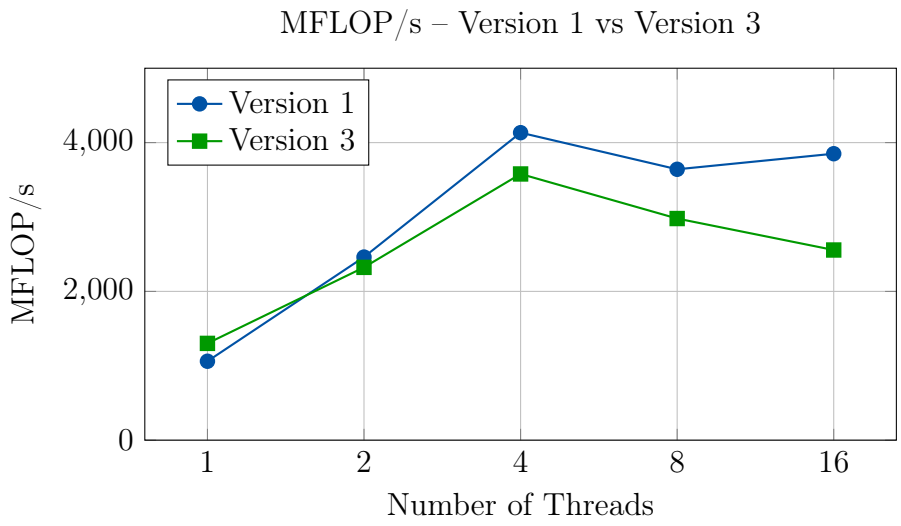

Figure 2: MFLOP/s comparison between Version 1 and Version 3

4.5 Discussion

4.5.1 Version 1 vs Version 3

Somewhat counterintuitively, Version 1 (with the implicit barrier) outperforms Version 3 (`nowait`) at every thread count except 1. At first glance one would expect `nowait` to be faster since it removes the synchronization cost. However, there is a subtle race condition in Version 3 as written: multiple threads are updating the same `lhs` array without any atomicity guarantee, since each column c contributes to all rows. When the barrier is removed, threads can overlap their writes to `lhs`, causing data races that corrupt the result and also hurt performance due to cache line contention (false sharing on `lhs`). Version 1 avoids this because the static schedule assigns disjoint columns to each thread, and the barrier ensures the result is coherent before use. Version 3's degraded performance at 8 and 16 threads is likely a consequence of this contention.

4.5.2 Why barriers limit scalability

A barrier forces all threads to wait until the slowest one finishes before any can proceed. In a perfectly balanced workload this overhead is small, but with any load imbalance the fast threads sit idle at the barrier. For this matrix-vector multiply with a static schedule, the work is evenly split (each thread handles n/p columns), so the barrier cost is relatively low – which is why Version 1 still scales reasonably well up to 4 threads.

Beyond 4 threads the speedup flattens and even drops slightly. With $n = 40000$ columns and $m = 600$ rows, the matrix is about $40000 \times 600 \times 8 = 192$ MB, which exceeds any reasonable L3 cache. As thread count increases, memory bandwidth becomes the bottleneck rather than compute, and adding more threads does not help once bandwidth is saturated.

4.5.3 When nowait becomes dangerous

`nowait` is safe only when the loop body writes to *disjoint* memory regions across threads, or when the result of the loop is not read until after all threads have finished anyway. In the `dmvm` case, removing the barrier is dangerous because `lhs[r]` is written by every

thread (different columns c all update the same row r). A thread could start a second loop iteration (or a subsequent computation) while another thread is still writing to `lhs`, producing incorrect results. In general, `nowait` should only be used when you can prove by inspection that no subsequent code reads data that may still be in flight.